



TEMA 10

WebStorage - Armazenamento no Navegador

[Conceitos](#)[12 questões](#)[localStorage | sessionStorage | JSON](#)

O que é WebStorage?



Analogia: WebStorage é como um **armário** dentro do seu navegador. Você pode guardar informações (chave/valor) e acessá-las sempre que visitar o site. É como ter um "caderninho" que o site usa para lembrar de você!

WebStorage é uma API do HTML5 que permite armazenar dados no navegador do usuário de forma **simples**, podendo ser **persistente** ou **temporária**. Ela oferece uma **alternativa** aos cookies para determinados tipos de armazenamento no navegador.



Exemplo do dia a dia: Quando você preenche um formulário e o site "lembra" seus dados na próxima visita, ele provavelmente está usando WebStorage!



Duas formas de armazenamento:

- `localStorage` → dados **persistentes** (não expiram)
- `sessionStorage` → dados **temporários** (associados à sessão da aba)



Vantagens sobre cookies:

- **Mais espaço** (cerca de 5MB vs 4KB dos cookies)
- **Não são enviados** automaticamente em requisições HTTP
- **API mais simples** e fácil de usar



localStorage vs sessionStorage

Característica	localStorage	sessionStorage
Persistência	Dados permanecem até serem removidos manualmente	Dados são apagados ao fechar a aba ou janela
Compartilhamento	Compartilhado entre todas as abas da mesma origem	Único por aba (cada aba tem seu próprio armazenamento)
Limite	Cerca de 5MB (pode variar entre navegadores)	Cerca de 5MB (pode variar entre navegadores)
Quando usar	Preferências do usuário, temas, dados que devem persistir	Formulários temporários, estado de cadastro, filtros de navegação



Dica de uso:

- Use `localStorage` para **preferências** (tema escuro, idioma)
- Use `sessionStorage` para **dados temporários** (formulários, estado de cadastro)



Métodos e Propriedades

Método/Propriedade	Descrição	Exemplo
setItem(chave, valor)	Salva um item no armazenamento	<code>localStorage.setItem("nome", "Ana")</code>
getItem(chave)	Recupera um item do armazenamento	<pre>let nome = localStorage.getItem("nome")</pre>
removeItem(chave)	Remove um item do armazenamento	<code>localStorage.removeItem("nome")</code>
clear()	Remove todos os itens do armazenamento	<code>localStorage.clear()</code>
length	Retorna o número de itens armazenados	<code>let total = localStorage.length</code>
key(índice)	Retorna o nome da chave pelo índice	<code>let chave = localStorage.key(0)</code>



Exemplos de Código para Copiar e Executar

Clique em " **Copiar**" para copiar o código ou " **Executar**" para abrir diretamente no OneCompiler.

1. localStorage - Operações Básicas

localStorage-basico.html

```
let cidade = localStorage.getItem("cidade");

output.innerHTML += `Nome: ${nome}<br>`;
output.innerHTML += `Idade: ${idade}<br>`;
output.innerHTML += `Cidade: ${cidade}<br>`;

// 3. CONTAR quantos itens estão salvos
output.innerHTML += `Total de itens: ${localStorage.length}<br>`;

// 4. REMOVER um item específico
localStorage.removeItem("cidade");
output.innerHTML += `<br>✅ Cidade removida!<br>`;

// 5. Verificar remoção
let cidadeRemovida = localStorage.getItem("cidade");
output.innerHTML += `Cidade após remoção: ${cidadeRemovida} (null)<br>`;

</script>
</body>
</html>
```

2. sessionStorage - Dados Temporários

sessionStorage.html

```
<!DOCTYPE html>
<html>
<head>
  <title>sessionStorage - Dados Temporários</title>
</head>
<body>
  <h2>📁 sessionStorage - Dados Temporários</h2>
  <div id="output"></div>

  <script>
    // =====
    // SESSIONSTORAGE - Dados Temporários
    // =====

    const output = document.getElementById('output');
```

```
// 1. SALVAR dados (apenas na aba atual)
sessionStorage.setItem("formulario", "Etapa 2 de 5");
```

✦ 3. Salvando Objetos com JSON

storage-objetos.html

```
<!DOCTYPE html>
<html>
<head>
  <title>JSON + localStorage</title>
</head>
<body>
  <h2>📦 Salvando Objetos com JSON</h2>
  <div id="output"></div>

  <script>
    // =====
    // SALVANDO OBJETOS COM JSON
    // =====

    const output = document.getElementById('output');

    // 1. Criando um objeto
    let usuario = {
      nome: "João Silva",
```

✦ 4. Versão Robusta com Try/Catch

storage-robusto.html

```
<!DOCTYPE html>
<html>
<head>
  <title>WebStorage - Versão Robusta</title>
</head>
<body>
  <h2>🛡️ WebStorage - Versão Robusta</h2>
  <div id="output"></div>

  <script>
    // =====
    // WEBSTORAGE - VERSÃO ROBUSTA COM TRY/CATCH
    // =====

    const output = document.getElementById('output');

    // Função helper para salvar com segurança
    function salvarSeguro(chave, valor) {
      try {
```

⚠ **IMPORTANTE:** O WebStorage só armazena **strings**. Para salvar objetos ou arrays, use `JSON.stringify()` para converter e `JSON.parse()` para recuperar. **Sempre verifique** se o dado existe antes de fazer o parse!

⚠ Limitações e Cuidados

● Limitações importantes:

- 🗝 **Segurança:** Dados são acessíveis por qualquer código da página
- 📦 **Tamanho:** Limitado a aproximadamente **5MB** (varia por navegador)
- 📄 **Apenas strings:** Use JSON para objetos e arrays
- ⌚ **Síncrono:** Opera de forma síncrona (pode bloquear a interface)
- 🔑 **Sem expiração:** Dados ficam até serem removidos manualmente
- ❌ **Sem proteção:** Dados não são criptografados

✅ Boas práticas:

- Não armazene **dados sensíveis** (senhas, dados bancários)
- **Não confie** em dados do WebStorage como prova de identidade ou autorização
- Use **nomes de chave descritivos** para facilitar a manutenção
- Sempre verifique se o dado existe antes de usar (com `if (dados)`)
- Use **try...catch** ao fazer parse de JSON

💡 **Dica:** Para ver os dados salvos no Chrome, pressione F12, vá em "Application" → "Storage" → "Local Storage" ou "Session Storage".

● ATENÇÃO - SEGURANÇA:

Nunca use WebStorage para controle de acesso ou autenticação. Por exemplo, **NUNCA** faça:

```
localStorage.setItem("admin", "true");  
// Depois confiar nisso para permitir acesso administrativo ❌
```

Isso é **inseguro** porque qualquer usuário pode alterar os dados do WebStorage diretamente no navegador!

Resumo do WebStorage

localStorage

Dados **persistentes** que não expiram

```
localStorage.setItem(chave, valor);
```

sessionStorage

Dados **temporários** (expira ao fechar a aba)

```
sessionStorage.setItem(chave, valor);
```

JSON.stringify()

Converte objeto em **string** para salvar

```
JSON.stringify(objeto);
```

JSON.parse()

Converte **string** em objeto para usar

```
JSON.parse(string);
```



Exercício de Fixação

Responda as **12 questões** sobre WebStorage.

Ao final, clique em "**Verificar Respostas**" para corrigir.

1 Qual método é usado para **salvar** um dado no localStorage?

☐ getItem()

☒ setItem()

☐ removeItem()

☐ clear()

2 Qual método é usado para **recuperar** um dado do localStorage?

☒ getItem()

☐ setItem()

☐ removeItem()

☐ clear()

3 Qual a principal diferença entre localStorage e sessionStorage ?

☐ localStorage é mais rápido

☒ localStorage mantém dados mesmo após fechar o navegador

☐ sessionStorage armazena mais dados

☐ Não há diferença

4 Qual método remove **todos** os itens do localStorage?

☐ removeItem()

☐ deleteItem()☒ clear()☐ erase()

5 Qual é o limite aproximado de armazenamento do localStorage?

☐ 1MB☒ 5MB☐ 10MB☐ 100MB

6 Analise o código:

```
localStorage.setItem("nome", "Ana");  
let valor = localStorage.getItem("nome");  
console.log(valor);
```

Qual será a saída no console?

☐ null☒ "Ana"☐ undefined☐ Erro

7 Qual método é usado para **remover** um item específico do localStorage?

☒ removeItem()☐ clear()☐ deleteItem()

☐ erase()

8 O que acontece com os dados do `sessionStorage` quando o usuário fecha a aba do navegador?

☐ Os dados são salvos permanentemente

☒ Os dados são apagados

☐ Os dados são movidos para o `localStorage`

☐ Os dados são enviados para o servidor

9 Qual função é usada para **converter um objeto em string** antes de salvar no `localStorage`?

☐ `JSON.parse()`

☒ `JSON.stringify()`

☐ `JSON.convert()`

☐ `JSON.format()`

10 O `localStorage` e o `sessionStorage` compartilham os mesmos dados.

☐ ☒ Verdadeiro

☒ ☐ Falso

11 O WebStorage só pode armazenar dados do tipo **string**.

☒ ☐ Verdadeiro

☐ ☒ Falso

12 **Analise o código:**

```
let usuario = { nome: "João", idade: 25 };  
localStorage.setItem("usuario", JSON.stringify(usuario));
```

```
let dados = localStorage.getItem("usuario");  
if (dados) {  
  let obj = JSON.parse(dados);  
  console.log(obj.nome);  
}
```

Qual será a saída no console?

☐ undefined

☒ "João"

☐ null

☐ Erro



Resultado do Quiz

12 / 12

Respostas corretas:

- ✓ Q1 - Correta
setItem() é usado para salvar um dado no localStorage.
- ✓ Q2 - Correta
getItem() é usado para recuperar um dado do localStorage.
- ✓ Q3 - Correta
localStorage mantém os dados mesmo após fechar o navegador, até que sejam removidos.
- ✓ Q4 - Correta
clear() remove todos os itens do armazenamento.
- ✓ Q5 - Correta
O localStorage tem limite aproximado de 5MB (pode variar por navegador).
- ✓ Q6 - Correta
getItem("nome") retorna o valor "Ana" que foi salvo.
- ✓ Q7 - Correta
removeItem() remove um item específico do armazenamento.
- ✓ Q8 - Correta
sessionStorage apaga os dados quando a aba é fechada.
- ✓ Q9 - Correta
JSON.stringify() converte um objeto em string JSON.
- ✓ Q10 - Correta
localStorage e sessionStorage são armazenamentos independentes.
- ✓ Q11 - Correta
O WebStorage só armazena strings, por isso usamos JSON.stringify().
- ✓ Q12 - Correta
O código verifica se dados existe, faz o parse e acessa obj.nome = "João".



PARABÉNS! Você acertou todas! Excelente domínio sobre WebStorage!



IMPORTANTE: Clique em "Imprimir" para salvar o resultado em PDF.